提示词工程与 LLM 应用

第 6 章

提示词工程 · 思维链 · 检索增强生成 · 智能体

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录

6.1 提示词工程 第 3 页

从任务描述到上下文学习：提示词如何激发预训练模型的能力

6.2 思维链 第 16 页

让模型先推理再作答：从推理问题的困难到 CoT 提示

6.3 CoT 扩展方案 第 27 页

自一致性、零样本 CoT 与思维树：从单条链到搜索

6.4 检索增强生成 第 41 页

用外部知识弥补幻觉与时效性：RAG、Self-RAG 与嵌入模型

6.5 智能体 第 53 页

规划、记忆与工具：LLM 作为智能体的大脑

6.6 自指令数据合成 第 63 页

用提示词生成指令数据：Self-Instruct 与 Alpaca

6.7 评估 第 75 页

基准测评、胜率、ELO 排位与大模型裁判

6.8 小结 第 86 页

两条路径、两个方向：本章的回顾

6.9 附录：In-Context learning 理论* 第 89 页

贝叶斯、梯度下降与信息论三个视角

阅读建议 先读提示词与思维链（6.1–6.3），再读 RAG、智能体与评估（6.4–6.7）。

6.1

提示词工程

不改动模型参数，只通过组织输入来引导模型；其有效性建立在上下文学习之上。

本节内容

6.1.1 概述
6.1.2 上下文学习
6.1.3 常用术语定义
6.1.4 效果

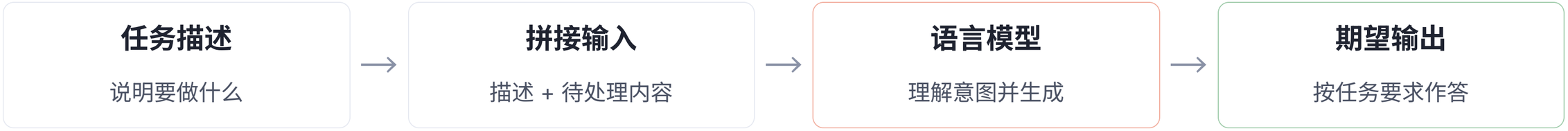
提示词工程的作用

在语言模型应用中，为了有效解决各类文本任务，通常需要提供明确的任务描述，并将其与任务输入拼接后送入模型，帮助模型准确理解用户意图。

任务描述：翻译中文到英文

任务输入：苹果

拼接后的提示词：翻译中文到英文{苹果}：



- 核心价值之一：解决零样本任务，让未经特定任务微调的预训练模型也能执行新任务。
- 有监督微调需要收集标注数据并调整模型参数；提示词不改参数，通过组织输入激发模型已有的能力。
- 提示词工程是一套系统性操作：与模型交互只是其中一步，还包括过滤、去重等常用数据处理方法。

上下文学习的两种方式

提示词工程的有效性建立在上下文学习（In-Context Learning, ICL）之上：通过优化输入提示的结构和内容，引导模型调整预测分布。

基于指令描述的优化

精心设计任务指令、约束条件或推理步骤，直接告诉模型要做什么、按什么格式做。

例如在指令中要求“先写出计算过程，再给出答案”，即思维链提示。

基于示例的学习

在提示词中提供少量输入-输出示例，供模型参考与模仿。

即少样本学习（Few-shot Learning）：示例不必训练，直接写在提示里。

> **核心思想** 输入中的上下文越完整、详尽、相关，模型对正确目标输出的条件概率预测就越准确。ICL 依赖预训练“预测下一个词元”所获得的基础推理能力，不进行梯度更新；指令负责界定任务，示例负责展示模式。

提示词工程的应用范围

文本

生成与理解

文本生成、翻译、摘要、文本分类与信息抽取。

推理

代码与数学

代码生成、数学推理、逻辑推理与定理证明。

外部信息

检索与工具

数据库交互（Text-to-DB）、检索增强生成（RAG）、网络搜索（Web Search），调用 API、计算器、搜索引擎。

系统

智能体与多模态

以 LLM 为核心的智能体；图文理解、图像生成与视频理解。

数据侧

数据合成、数据标注。

评测侧

模型测评与自动评判。

风险侧

提示注入（Prompt Injection）与越狱（Jailbreaking）。

> **注** 提示词工程降低了应用门槛；但被恶意构造的提示也可能诱导模型产生有害或不安全的输出。

提示词的原理

预训练模型已经蕴含了解决问题的知识与推理能力；提示词工程的作用是有效激发这些内在能力，引出所需答案。

认知转变

范式改变

提示词范式揭示了模型在提示引导下展现的涌现能力，改变了“一个任务训练一个模型”的传统认知。

能力来源

知识已在内

通用表征与预测能力来自大规模预训练；提示只是把已有的知识组织成可用的形式。

下一步

自主决策

更具前景的方向是让模型在开放环境中自主决策与学习，借鉴强化学习的经验学习，通过与环境交互自我改进。

小结 提示词工程是“外部优化”：不改动 θ，而是设计输入 $\boldsymbol{x}$，把预训练模型从“会预测下一个词元”推进到“能按意图完成通用任务”。

GPT-2 的零样本能力

GPT-2 的研究表明：预训练语言模型仅需在输入中提供明确的任务描述，即可在未经任务特定微调的情况下执行多种任务。

传统任务特定模型

在独立同分布（i.i.d.）数据上表现良好，但在分布外（OOD）测试集上性能显著下降。

需要收集大量带标注的数据做有监督微调，数据标注与计算成本高昂。

每个新任务都要重复一遍数据收集与训练。

大规模预训练模型

在不同领域的海量无监督语料上训练，获得强大的泛化能力。

即使没有监督数据、没有示例，仅凭任务描述也能完成新任务。

这就是零样本学习（Zero-shot Learning）能力。

> **对照** 前者把任务知识写进参数，后者放进输入。输入中的任务描述本身就足以调动模型已学到的语言能力。

零样本、单样本与少样本

GPT-3 系统拓展了 ICL：按提示中示例的数量，分为零样本、单样本与少样本三种设置。翻译任务的三种提示如下。

零样本

只给任务描述

```
问题：翻译中文到英文：
苹果 >
回答：
```

仅依靠指令或问题描述。

单样本

加入一个示例

```
问题：翻译中文到英文：
香蕉 > banana
苹果 >
回答：
```

示例格式与目标问题相同。

少样本

加入少量示例

```
问题：翻译中文到英文：
香蕉 > banana
桃子 > peach
橙子 > orange
苹果 >
回答：
```

通常 K 个示例，$K > 1$。

推理方式 模型从同类型任务的示例中进行类比推理：识别“左边是中文、右边是英文”的模式，再把它应用到目标问题上。

少样本学习的优势

- 用户只需在提示词中提供少量任务示例，即可引导模型解决同类型的新问题。
- 若改用有监督微调，则需额外承担收集标注数据、训练专用模型的计算成本，且每个独立任务都要重复一遍。
- 精心构建示例后，模型可灵活应对跨领域任务：常规的翻译、分类、摘要，以及求解数学问题、生成代码、证明数学定理等复杂认知任务。

关键观察

规模与 ICL 性能强关联

GPT-2 与 GPT-3 展现的零样本与少样本泛化能力，明显超越同期其他方法，验证了扩展模型规模与上下文学习性能提升之间的强关联。

理论意义

预训练-提示范式

这一发现标志着预训练-提示范式在通用语言理解与生成系统上的突破，揭示了模型涌现能力的潜力，为后续大语言模型的发展奠定了理论基础。

理论机制（贝叶斯、梯度下降、信息论三个视角）见本章附录 6.9 节。

提示词的组成

整体

提示词（Prompt）

输入给语言模型的整体内容，可能包含问题、指令、上下文信息或其他引导性文本。

角色设定

系统提示词（System Prompt）

设定模型角色、功能或约束回复风格的指令性文本，通常位于输入的开始部分。

信息获取

问题（Question）

用户寻求信息或解答的询问，如“宇宙起源是什么？”。

任务执行

指令（Instruction）

要求模型执行特定任务的明确请求，如“总结以下文章：...”“使用 Python 实现：...”。模型遵循指令的能力称为指令跟随能力。

上下文 为模型理解或执行任务提供的相关信息：可以是问题中明确指出的内容，也可以是背景说明或历史对话记录。对话模板（Chat Template）把这些元素格式化为统一的交互信息。

提示词工程与参数优化的区别

对一个参数为 θ 的概率语言模型，其预测分布为 $p_\theta(\cdot \mid \boldsymbol{x})$。改变输出分布有两条途径。

$$p_\theta(\cdot \mid \boldsymbol{x}) \quad \longrightarrow \quad p_\theta\left(\cdot \mid \boldsymbol{x}'\right), \qquad \boldsymbol{x}' \leftarrow (\mathbf{prompt}, \boldsymbol{x})$$

提示词工程把原输入 $\boldsymbol{x}$ 扩展为 $\boldsymbol{x}'$，在参数 θ 不变的前提下改变条件分布。

内部优化：调整 θ

通过微调、指令微调等手段改动模型参数，把任务能力写进权重。

成本高，但推理时不增加输入长度。

外部优化：调整 $\boldsymbol{x}$

不改参数，通过提示词引入新的引导信息，改变模型的条件概率分布。

灵活、可即时修改，是提示词工程的核心。

提示方法分类

不提供示例

提示词学习

通过清晰的任务描述、指令或辅助信息引导模型输出，不提供任务示例。

零样本场景的常用方法，是上下文学习的一种特例。

提供示例

上下文学习

在输入中提供输入-输出对，引导模型模仿示例的模式。

按示例数量分为零样本、单样本、少样本；少样本通常给 3–5 个示例。

训练方式

指令微调

用包含多种任务指令及期望输出的数据集做有监督微调，获得通用的指令跟随能力。

训练方法与常规 SFT 一致，区别在于指令数据具备多样性。

具备良好上下文学习能力的模型称为**少样本学习器**（Few-shot Learner），这一称呼不限定模型是否经过有监督微调。

经过指令微调的模型通常更适合零样本或单样本方式：主要依赖清晰的指令，而非大量示例。

推理技术

思维链提示（CoT）

示例中包含逐步推导的推理过程，引导模型在解决类似问题时也先生成中间步骤，再得出最终答案，从而提高复杂问题解答的准确性。

GPT-3 的上下文学习效果

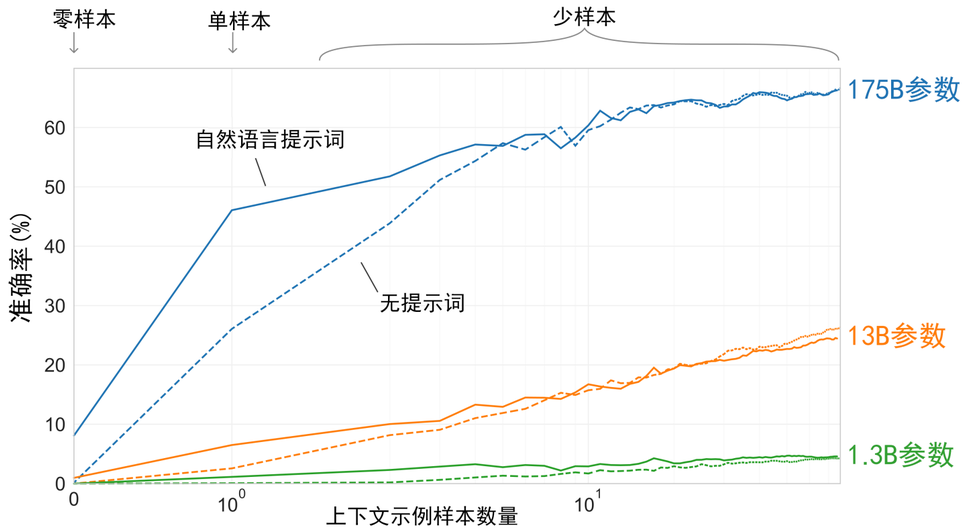


图 6.1　GPT-3 模型上下文学习效果

- 在“词序重组或扰动”任务中，模型需要识别被打乱的词汇并重建原句。
- 175B 参数的 GPT-3 在零样本与单样本设置下，使用自然语言示例的提示即可带来显著提升。
- 示例增至 10 个时，准确率相比单样本设置提高 15%。
- 即使不加提示词，仅增加测试时的上下文示例，性能也能接近使用提示词的设置。

横轴为上下文示例数量，纵轴为任务准确率；曲线随示例数上升并趋于平缓。

图源 《大语言模型：训练、精调与对齐》图 6.1；**文献** Brown et al., *Language Models are Few-Shot Learners*, NeurIPS 2020

涌现能力

研究者进一步发现：当预训练语言模型的规模超过某个临界点，会在多个任务上表现出“涌现”现象。

理解

文本理解

阅读、归纳与推断类任务。

转换

翻译

跨语言的语义对齐与生成。

计算

算术

多步四则运算与数量推理。

知识

问答

开放域与专业知识问答。

- 多个模型在参数量超过特定规模后，在上述任务的测评中性能获得显著跃升。
- 模型仅需通过少样本学习即可有效完成任务，无需针对特定任务微调，展现出强大的通用文本处理能力。
- 这一现象是“规模扩展带来能力跃迁”的直接证据，也是提示词范式被广泛接受的基础。

依据 《大语言模型：训练、精调与对齐》第 6.1.4 节；**文献** Wei et al., *Emergent Abilities of Large Language Models*, TMLR 2022

6.2 思维链

对需要多步推理的问题，展示推理过程比只给答案更有效。

本节内容

6.2.1 推理问题
6.2.2 思维链
6.2.3 CoT 的性能表现

多步推理任务

涉及推理、数学、代码生成与解密的语言任务，往往需要多步推理才能得到正确答案。这类问题通常具有客观正确的答案，而得出答案的过程需要逻辑推理。

形式一

选择题问答

```
问题：小明手里有两袋苹果，每个袋
有3个苹果，小明给了小强一个苹果，
请问现在小明手里有几个苹果？
（a）3个　（b）1个
（c）4个　（d）5个
回答：
```

形式二

问答形式

```
问题：小明手里有两袋苹果，每个袋
有3个苹果，小明给了小强一个苹果，
请问现在小明手里有几个苹果？
回答：
```

同样的题面，去掉选项，要求自由作答。

模型可以基于生成能力直接预测答案；但在零样本条件下，模型通常只输出最终答案，不展示也不隐含得出答案的推理过程。

示例提示的局限

只有答案的示例

上下文学习

```
问题：小明有10个苹果，给了小强
一半，然后小明吃了1个苹果，最终
小明剩下多少苹果?
回答：4个
问题：小明手里有两袋苹果，每个袋
有3个苹果，小明给了小强一个苹果，
请问现在小明手里有几个苹果?
回答：2个（错误）
```

带推理的示例

结构化解答

```
两个袋子，每个袋子有3个苹果，
那么就有 2×3=6，现在小明有6个
苹果；小明给了小强一个苹果，那么
就是 6−1=5，现在小明有5个苹果，
所以：
答案：5个
```

- 关键原因：示例本身缺乏详细的推理步骤，模型难以学习到问题与答案之间的潜在逻辑联系，推理准确率仍然较低。
- 人类解答这类问题通常会遵循结构化的推理步骤，因此示例也应展示这些步骤，而不只是答案。

结构化推理

结构化推理不只适用于算术题，它也是解决众多实际复杂问题的关键。

读懂条件
提取已知量与约束

→

分步推导
每步只做一个运算或判断

→

核对中间结果
检查是否偏离目标

→

得出答案
给出结论并说明依据

合同

条款推导

根据合同条款推导理赔金额：定位适用条款，逐条判断条件，累计赔付。

数据

数据库统计

通过数据库查询统计年销售额：拆解指标口径，写出查询，核对结果。

数学

定理证明

从已知条件出发，逐步施加推理规则，直到得到待证结论。

高效运用提示词工程与上下文学习，引导模型生成清晰的思维链，是提升复杂推理任务表现的关键。

讨论

思考题

如果模型给出的思维链中**有一步推理是错的**，但最终答案恰好正确，这个回答能否被接受？

思维链提示

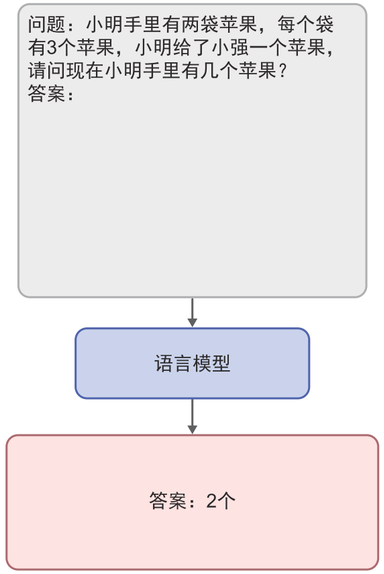
对数学、代码和推理等逻辑性较强的问题，模型难以仅通过示例进行有效学习。思维链（Chain-of-Thought, CoT）提示在上下文示例中显式展示包含推理步骤的解答过程。

```
问题：小明有10个苹果，给了小强一半，
然后小明吃了1个苹果，最终小明剩下
多少苹果?
回答：小明10个苹果，一半苹果为
10/2=5，给了小强一半苹果，剩下
10−5=5 个苹果，小明吃了一个苹果，
5−1=4 个苹果，最终小明剩下4个苹果。
问题：小明手里有两袋苹果，每个袋有
3个苹果，小明给了小强一个苹果，请问
现在小明手里有几个苹果?
回答：
```

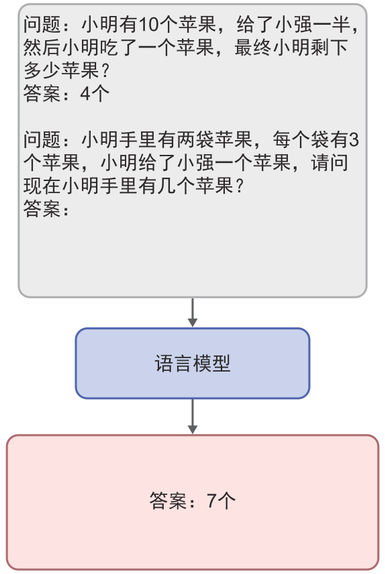
- 示例不只给答案，还给出“怎么算出来的”。
- 模型借鉴示例的推理路径，在新问题上生成结构化输出。
- 示例中的每一步都可被模型当作模板复用，例如“先算总数，再减去给出的数量”。

> **作用** CoT 通过展示逻辑严密的推理过程，引导（elicit）模型进行结构化思考。

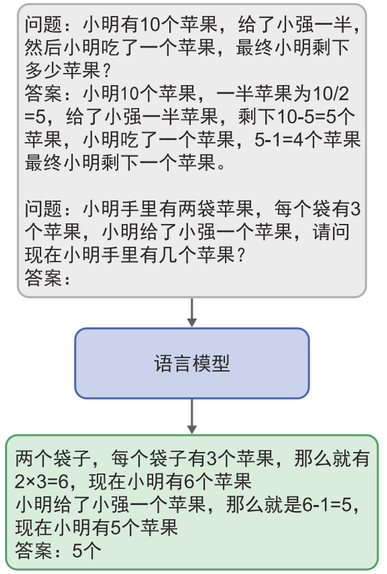
三种提示方式的对比



(a) 零样本学习：直接作答，答案错误



(b) 单样本学习：只给答案示例



(c) CoT 提示词：示例含推理步骤，答案正确

图 6.2　CoT 提示词可以提高数学问题的解答准确性

图源 《大语言模型：训练、精调与对齐》图 6.2 (a)(b)(c)

思维链的适用范围

CoT 作为上下文学习的重要技术，已被广泛应用于各类推理任务。

适用

何时用 CoT

任务需要多步推导、且中间步骤可以自然写成文字时效果最好：算术应用题、逻辑推理、代码生成、符号运算。

若任务只是简单检索或格式转换，长推理反而增加成本与出错面。

演进

从链到树与图

在 CoT 基础上，研究者提出了思维树（Tree of Thought, ToT）与思维图（Graph of Thought, GoT）等拓展方案。

6.3 节将展开自一致性、零样本 CoT 与思维树。

CoT

单条推理链

Self-Consistency

多条链投票

Zero-shot CoT

一句提示即触发推理

ToT / GoT

树状与图状搜索

GSM8K 数据集

GSM8K 是由 OpenAI 发布的小学数学推理数据集，用于检验模型的多步算术能力。

8 500
道高质量、语言多样的题目

7 500
训练集题目

1 000
测试集题目

2–8
步推理，主要用四则运算

- 题目需要按顺序进行多步计算，最终答案是一个数值。
- 难度相当于小学高年级水平：难点不在运算本身，而在于把应用题正确拆解为若干步。
- 正因为步骤明确、答案唯一，GSM8K 成为比较不同提示方法的标准试验台。

PaLM 540B 上的实验结果

18%

标准提示词在 GSM8K 上的准确率

57%

使用 CoT 提示词后的准确率

在 PaLM 540B 模型上测试；性能提升超过三倍。

提示词输入

```
Q: Roger has 5 tennis balls. He buys
2 more cans of tennis balls. Each can
has 3 tennis balls. How many tennis
balls does he have now?
A: Roger started with 5 balls. 2 cans
of 3 tennis balls each is 6 tennis balls.
5 + 6 = 11. The answer is 11.
Q: The cafeteria had 23 apples. If they
used 20 to make lunch and bought 6
more, how many apples do they have?
A:
```

模型的推理输出

面对同一道苹果题，模型按示例的格式先逐步计算，最后给出答案。推理过程本身成为可检查的中间结果。

> The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. **The answer is 9.**
>
> 模型输出：先算剩余，再算补充，最后给出结论。示例中的“先…再…最后…”结构被完整模仿。

- 算术步骤 $23 - 20 = 3$、$3 + 6 = 9$ 都可独立核对；哪一步错了一眼可见。
- 因此 CoT 不仅提升准确率，也让答案具备可解释性与可验证性。
- 示例的写法决定了输出的形式：示例分步，输出就分步；示例只给答案，输出也倾向于只给答案。

6.3

CoT 扩展方案

CoT 只生成一条链；扩展方案分别从采样、示例构造与搜索三个方向改进它。

本节内容

6.3.1 自一致性
6.3.2 零样本 CoT
6.3.3 思维树
6.3.4 其他方法

思维链的三类扩展

在 CoT 方法基础上，研究者提出了多种扩展方案以提高性能，分别针对 CoT 的三个薄弱环节。

针对随机性

自一致性（Self-Consistency）

通过增加测试时的采样数量来提升最终答案的确定性。

多条推理路径，答案投票。

针对人工设计

零样本 CoT（zero-shot CoT）

用一句通用提示“让我们一步步思考”替代任务特定的示例，激发模型的逐步推理。

免去逐任务编写示例。

针对单链错误

思维树（ToT）

把链式推理扩展为树状结构，通过搜索可能的推理步骤序列来解答问题。

前提：问题能被有效地分解并形成搜索空间。

关系 三者并非替代关系：自一致性仍以 CoT 为基本单元，零样本 CoT 解决示例从哪来，ToT 则改变了推理的组织形式。

自一致性

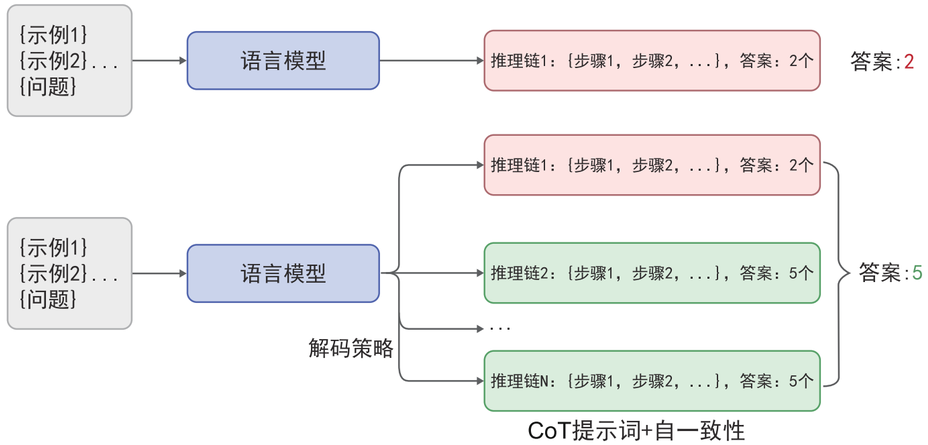


图 6.3　自一致性提高推理答案的准确性

- 标准 CoT 加入提示后常只生成一条思维链，取一个最终结果。
- 自一致性采用基于采样的解码方式，生成多条不同的推理路径，每条对应一个最终答案。
- 统计所有答案中出现频率最高的答案作为最终输出。

> **效果**　相比输出单条链的标准 CoT，自一致性能够缓解生成过程中的随机性错误；在 GSM8K 上相比标准 CoT 基准提升 **17.9%**。

图源 《大语言模型：训练、精调与对齐》图 6.3；**文献** Wang et al., *Self-Consistency Improves Chain of Thought Reasoning*, ICLR 2023

自一致性的原理

同一提示

问题 + CoT 示例

→

采样解码

温度 > 0，生成 N 条路径

→

提取答案

每条链末尾的最终结果

→

多数投票

出现频率最高的答案

标准 CoT：一次采样

只生成一条链。若中途某步算错，错误沿链传播，最终答案错误，且没有纠正机会。

结果受单次采样的随机性支配。

自一致性：多次采样 + 投票

正确路径往往收敛到同一个答案，而错误路径各有各的错法，票数分散。

代价是推理次数与计算开销成倍增加。

依据 《大语言模型：训练、精调与对齐》第 6.3.1 节

零样本 CoT（zero-shot CoT）



图 6.4　零样本 CoT 提示词

- 人工设计高质量的 CoT 提示示例需要反复试错，且往往只适用于特定任务。
- 零样本 CoT 不需要任何示例，只需在问题后追加一句通用提示：“让我们一步步思考”(Let's think step-by-step)。
- 核心优势：避免人工编写任务或问题类型特定的提示词，使模型更泛化地进行逐步推理。
- 标准少样本 CoT 则需要为不同类别的问题人工设计特定的提示示例。

> **效果**　同一句提示可用于算术、符号与逻辑推理等多样任务：在 GSM8K 上，text-davinci-002 的准确率由标准零样本的 **10.4%** 提升到 **40.7%**；在 MultiArith 上由 17.7% 提升到 78.7%。它仍略低于精心设计示例的少样本 CoT，收益来自通用性。

图源 《大语言模型：训练、精调与对齐》图 6.4；**文献** Kojima et al., *Large Language Models are Zero-Shot Reasoners*, NeurIPS 2022

零样本 CoT 与少样本 CoT 的对比

标准少样本 CoT

为每一类问题手工编写带推理步骤的示例，再拼进提示词。

示例质量依赖设计者的经验；换一类问题就要重写一遍。

前期投入高，跨任务迁移差。

零样本 CoT

用一句通用的“让我们一步步思考”触发逐步推理，不依赖任务特定的示例。

同一套提示可迁移到算术、常识、符号推理等多种任务。

实现简单，接近少样本 CoT 的效果。

对比项	标准少样本 CoT	零样本 CoT
示例来源	人工编写，逐个任务设计	无需示例，一句通用提示
任务特异性	强，换任务需重写	弱，可跨任务复用
GSM8K 准确率	接近零样本 CoT，设计成本高	**40.7%**（标准零样本 10.4%）

依据 《大语言模型：训练、精调与对齐》第 6.3.2 节

单链推理的问题

用 CoT 示例对问题做提示时，推理过程中若某一步推断错误，就可能影响后续推理，增大最终输出错误结果的概率。

CoT

一条路走到黑

生成一条链，中间步骤无法回退，也没有分支可以比较。

错误一旦发生，只能等最终答案暴露问题。

自一致性

多条路，但仍不看中间

生成多条路径，根据最终答案的一致性判别结果。

中间步骤出错不一定导致最终答案出错：有些错误会被后续步骤抵消或绕过。因此投票能缓解随机性，却不检查推理过程本身。

共同缺口 两者都不评估“当前这一步是否合理”。若能在每一步生成多个候选、即时评估并决定是否继续，就能在错误扩散之前把它截住。

思维树

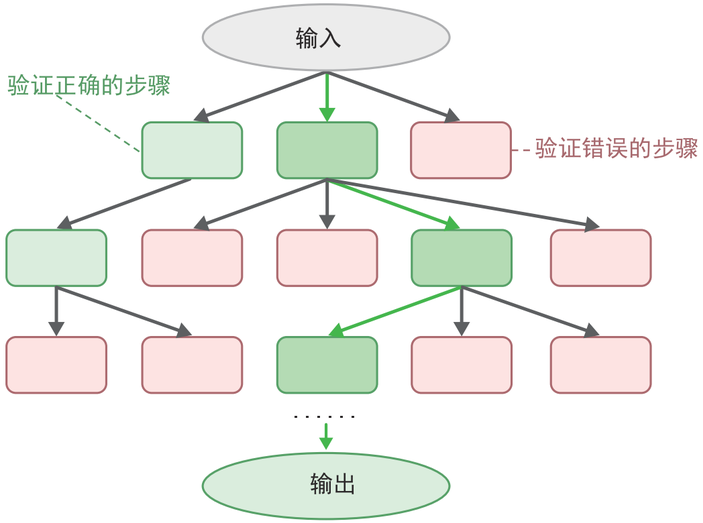


图 6.5　思维树的核心思想示意图

阶段一

思维生成

基于当前问题和过程解答状态，生成多个可能的后续推理步骤的候选节点。

阶段二

思维评估

评估每个候选节点执行后的新状态是否合理，以及引导至最终目标的潜力。

绿色为验证正确的步骤，红色为验证错误的步骤：错误节点不再扩展，回退到上一层另选分支。

图源 《大语言模型：训练、精调与对齐》图 6.5；**文献** Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, NeurIPS 2023

思维树的两个阶段

思维生成

Thought Generation

不是仅生成单一的下一步骤，而是预测多个候选步骤。

两种生成方式：**链式思维（CoT）提示**适用于输出空间较大的任务（如段落生成）；**提案（Proposal）提示**适用于输出空间受限、规则明确的任务（如 24 点）。

思维评估

Thought Evaluation

判断新状态是否合理，以及该节点引导至最终目标的潜力有多大。

评估方式：**自验证**（模型评估自己生成的步骤）或**自投票**（模型在多个选项中选择）；评估器可与生成器是同一个模型，也可以是不同的模型。

> **要点** 仅当评估认为节点合理时才基于它继续扩展；若不合理则回退并尝试其他候选，避免在错误方向上浪费资源。评估器换用不同模型，可提高评估的鲁棒性与一致性。

这一过程模拟了人类多步思考中不断探索、评估和回溯的决策过程。

思维树的完整流程

目标：用 $(4, 9, 10, 13)$ 四个数字，各用一次，经 $+, -, \times, \div$ 与括号得到 24。

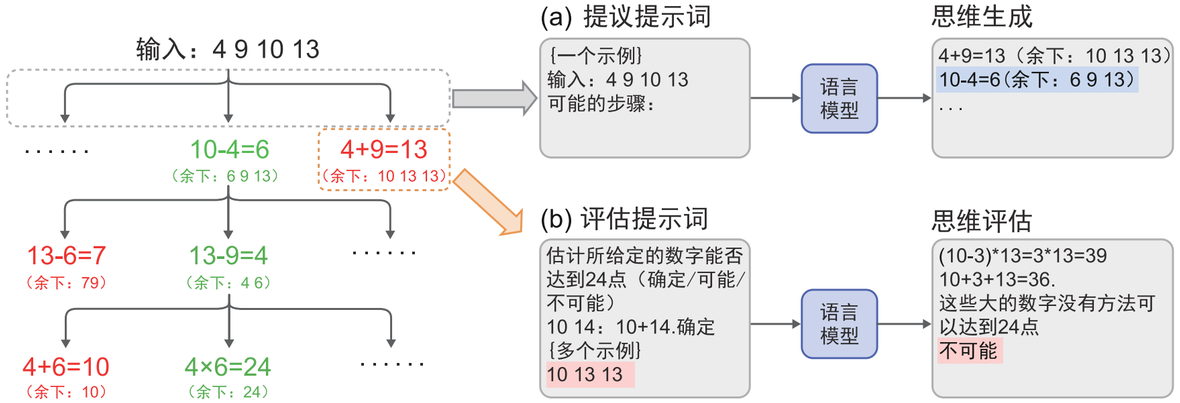


图 6.6　24 点游戏的思维树推理过程图（左为搜索树；右侧 (a) 为提案提示词与思维生成，(b) 为评估提示词与思维评估）

图源 《大语言模型：训练、精调与对齐》图 6.6；**文献** Yao et al., *Tree of Thoughts*, NeurIPS 2023

思维树的求解示例

状态

4, 9, 10, 13

→

$13 - 9 = 4$

剩余 4, 10, 4

→

$10 - 4 = 6$

剩余 4, 6

→

$4 \times 6 = 24$

剩余数字为空，达成目标

思维生成示例

输入状态 4, 9, 10, 13；要求生成所有可能且合理的下一步计算，格式为“操作 → 结果（剩余数字）”。

候选：$13 - 9 = 4$（剩余 4,4,10）；$10 - 4 = 6$（剩余 6,9,13）；$4 + 9 = 13$（剩余 13,10,13）。

思维评估示例

候选 $4 + 9 = 13$ 产生状态 [13,10,13]。逐一尝试运算均无法得到 24，评估为“不可能”，终止该节点的后续扩展。

回退到上一状态，改选 $10 - 4 = 6$ 得到 [6,9,13]，评估为“有可能”，继续生成与评估。

思维树的搜索策略

上述过程本质上是在一个树状结构的状态空间上进行搜索。

策略

深度优先

沿一条分支走到底，失败再回溯。

策略

广度优先

逐层扩展，比较同层候选后择优。

策略

蒙特卡洛树搜索

按模拟结果平衡探索与利用。

策略

A* 算法

用启发式估计剩余代价，优先扩展有希望的节点。

- 思维生成与思维评估主要依赖 LLM 的上下文学习能力，通过精心设计的提示词以零样本或少样本方式引导模型完成。
- 对模型做特定任务的微调可进一步提升性能，但 ToT 的核心机制并不强制要求微调。
- 对于非客观性问题，可以通过生成多个验证结果并采用投票机制来决定节点的扩展。

> **选择** 分支越多、搜索越深，答案质量的上限越高，但调用次数也成倍增长；24 点游戏中 $b = 1$ 已接近 Best-of-100，$b = 5$ 才把成功率推到 74%。

思维树的成功率

表 6.1　24 点游戏成功率（基于 GPT-4）

方法	成功率
问答提示词	7.3%
CoT 提示词	4.0%
CoT-SC（自一致性，采样 100 条路径）	9.0%
CoT（Best-of-100）	49%
ToT（$b = 1$，广度优先搜索）	45%
ToT（$b = 5$）	74%

- ToT 即使 $b = 1$（计算成本较低），性能（45%）也接近成本高得多的 Best-of-100 CoT（49%）。
- 增加搜索宽度 $b = 5$ 显著提升至 74%，远高于 CoT-SC 与 Best-of-K CoT。

依据 《大语言模型：训练、精调与对齐》表 6.1；**文献** Yao et al., *Tree of Thoughts*, NeurIPS 2023

自我前缀提示

在大语言模型提示词综述中，系统介绍了 29 种提高模型表现的方法，并整理出 12 种应用类别。

Self-Improve

自提升

模型基于自身输出迭代改进，逐步修正不准确的推理。

Self-Refine

自精炼

生成初稿后自我反馈、自我修订，反复打磨同一结果。

Self-Evaluation

自我评估

模型对自己的候选输出打分，用于选择性生成。

共同点 这些研究都涉及 LLM 的自我评估与更新，以此提高推理能力、减少幻觉。它们仍建立在规则设定好的处理流程之上。

另一条路径

DeepSeek-R1：把推理内化进参数

前述方法都在规则下设定处理流程，使模型产生特定的推理模式；DeepSeek-R1 则让模型自由推理，并通过强化学习优化参数，将推理能力内化至模型参数中（详见第 8 章）。

依据 《大语言模型：训练、精调与对齐》第 6.3.4 节；**文献** Vatsal et al., *A Survey of Prompt Engineering Methods*, 2024；DeepSeek-AI, *DeepSeek-R1*, 2025

6.4

检索增强生成

模型的知识固定在参数里；RAG 在生成前先检索，把外部证据放进上下文。

本节内容

6.4.1 检索增强生成流程
6.4.2 自反思检索增强生成
6.4.3 嵌入模型原理

幻觉的成因

语言模型幻觉指模型生成看似真实、实则为错误或虚构的信息。面对模糊输入或缺少相关知识时，模型可能依据训练数据中的相关性生成看似合理实则错误的输出。

类型一

事实性幻觉

提供虚假事实，如编造不存在的文献、人物或数据。

类型二

逻辑性幻觉

推理过程出现错误，结论与前提不成立。

类型三

语义性幻觉

语义看似合理，但内容并不成立。

- 模型本质上是基于概率生成，且训练数据难免包含噪声或知识盲区，幻觉难以彻底根除。
- 幻觉会误导用户决策、传播虚假信息，影响科学研究或医疗咨询，需要通过改进模型、加强验证机制、提升用户对模型局限性的认知来降低影响。
- 训练成本高昂导致模型无法实时更新，在时效性敏感场景下难以提供准确回复；基于网络搜索可用公开信息补足，但对私有化部署的重要数据无能为力。

检索增强生成的四个步骤

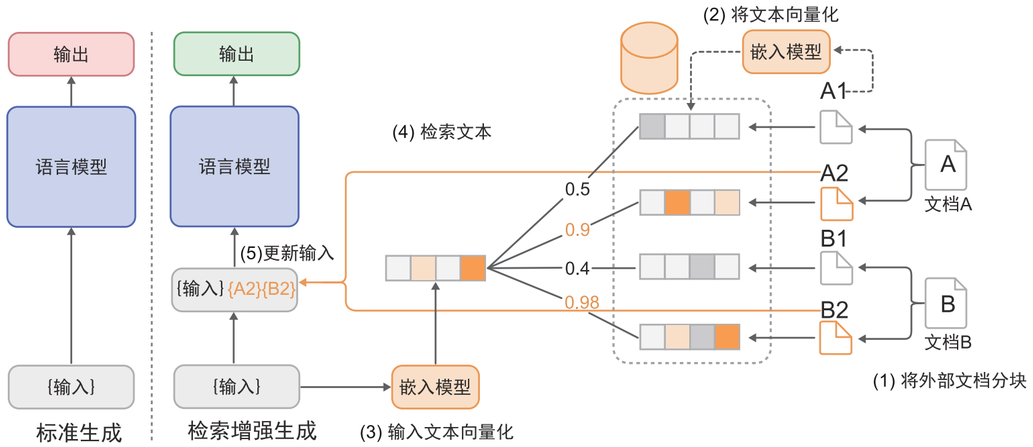


图 6.7　检索增强生成示意图

① 分块

按固定词元数切分私有文本

② 向量化

嵌入模型编码，存入向量库

③ 检索

查询向量比对，取 Top-K

④ 增强生成

上下文 + 查询一起送入模型

左侧为标准生成（输入直接进模型），右侧为检索增强生成：文档先分块、向量化，检索结果与输入拼接后再生成。

图源 《大语言模型：训练、精调与对齐》图 6.7；**文献** Gao et al., *Retrieval-Augmented Generation for Large Language Models: A Survey*, 2024

检索增强生成的细节

步骤 1

分块（Chunking）

将私有文本数据按固定词元数（如 512 或 1024 个词元）分割成块。

需注意：切分会导致信息不连续，关键信息可能被截断在块边界上。

步骤 2

向量化（Embedding）

用嵌入模型把各数据块转化为向量，存入向量数据库。

核心优势是提取文本语义特征、形成稠密向量表示，便于高效相似度检索。嵌入模型通常基于 Transformer 编码器架构。

步骤 3

检索（Retrieval）

对用户输入使用同一嵌入模型得到查询向量，计算其与文档库中向量的相似度。

返回相似度最高的 Top-K 个文档块作为上下文。

步骤 4

增强生成（Augmented Generation）

将检索到的上下文与原始查询一同输入语言模型，引导模型基于该上下文生成最终答案。

答案的依据来自检索到的文档，而不是参数记忆。

整条链路的关键约束：查询与文档必须由同一个嵌入模型编码，向量空间才可比。

检索增强生成的挑战

RAG 是有效利用外部知识提升回复表现的系统，但在实际应用中面临多重挑战。

挑战一

查询意图识别

并非所有问题都需要检索外部知识。

如何准确判断用户查询是否需要触发 RAG，是首要挑战。

挑战二

检索相关性

检索到的文档块若与查询语义相关性低，就失去了作为有效上下文的意义。

挑战三

生成质量依赖

即使检索到相关文档，生成的答案仍可能不正确或不完整。

原因有二：检索文档本身信息不足或有误；语言模型自身的推理能力有限。

挑战四

迭代检索策略

首次生成的答案无效或信息仍不足时，是否需要、以及如何进行多轮检索以获取更有效的信息。

上下文长度：过长的检索文本与多轮对话历史极易超出最大上下文窗口，既考验长文本理解，也会显著降低生成速度。

Self-RAG

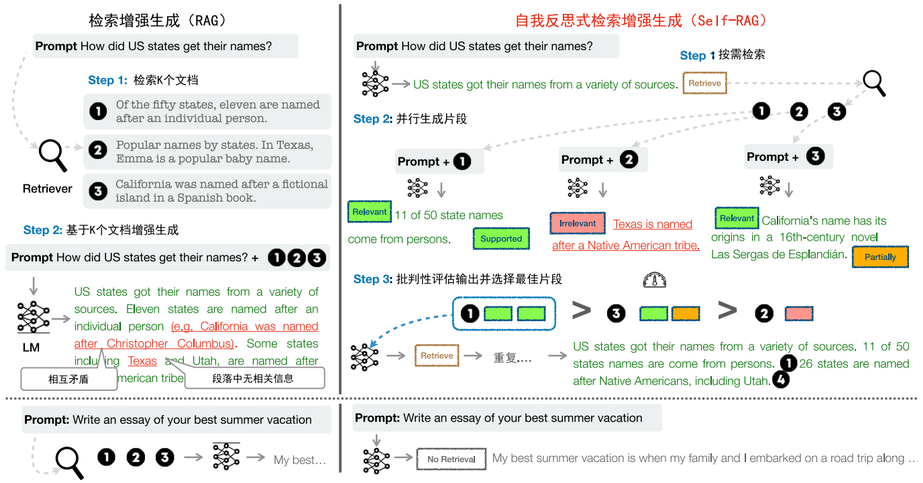


图 6.8　自反思检索增强生成示意图

Self-RAG 的核心思想：在生成过程中进行自我反思，动态地决定是否需要检索外部知识，并对检索到的信息和自身的生成内容进行批判和判别。

与固定流程的 RAG 相比，它把“检索—生成”变成带判断的决策过程：先判断要不要查，再判断查到的文档是否相关，最后判断答案是否被文档支持、是否真正有用。

> **代价**　Self-RAG 通过决策机制显著提升了生成准确性，但会增加耗时。

图源 《大语言模型：训练、精调与对齐》图 6.8；**文献** Asai et al., *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*, ICLR 2024

Self-RAG 的反思词元

目标模型会根据上下文，动态预测特殊词元，用它们承载“要不要检索、是否可信”的判断。

检索词元

[Retrieve]

是否需要进行检索。概率超过阈值则触发检索。

相关性词元

[IsRel]

检索到的文档与查询是否相关。

支撑性词元

[IsSup]

生成的答案是否被检索文档支持。

有用性词元

[IsUse]

答案是否有效解决了用户的问题。

> **训练方式** 执行 Self-RAG 的模型需要具备反思能力与预测特殊词元的能力，可通过一个批判模型（Critic Model）合成数据，用于训练目标模型。

四个词元把“检索质量”与“生成质量”拆成可分别预测的信号，从而解耦标准 RAG 的固定流程。

Self-RAG 的流程

① 检索必要性判别

[Retrieve] 概率超阈值才检索；能可靠回答则直接生成

→

② 相关性验证

并行验证文档与查询的相关性 [IsRel]

→

③ 并行生成与评估

对通过验证的文档生成候选答案，评估 [IsSup]、[IsUse]

→

④ 答案择优

综合三项评分，选择最优答案

- 检索必要性判别：若语言模型自身能可靠回答，则跳过检索流程，直接生成答案。
- 相关性验证：获取相关文档后，先并行验证其与查询的相关性。
- 并行生成与评估：评估每条答案的支持性（是否被检索文档支持）与有用性（是否有效解决问题）。
- 答案择优：综合相关性、支持性和有用性评分，选择最优答案输出。

> **可迭代** 若模型评判检索生成的结果不合理，可能再次预测出 [Retrieve] 并重新执行检索流程，从而动态获取外部信息、增强结果的可靠性。

句向量的表示

对于文本匹配问题，传统方法（字符级匹配或词频统计）可以在字符层面判断，但难以捕捉语义层面的相似性。

字面匹配

字符、词频统计

→

词元向量

Word2Vec 等实现词元向量化

→

段落向量

整段文本映射为固定维度表征

- 对于段落或句子级别的文本，需要将其整体转化为一个固定维度的表征向量，才能做相似度检索。
- 在 RAG 中，文档向量化是关键步骤：通常使用基于纯编码器架构的预训练 Transformer 模型（如 BERT）。
- 具体做法：把文本段落（视为一个序列）输入 BERT，对序列中所有词元的最终隐含层状态向量做平均池化，作为该段落的嵌入向量表征。

依据 《大语言模型：训练、精调与对齐》第 6.4.3 节

Sentence-BERT

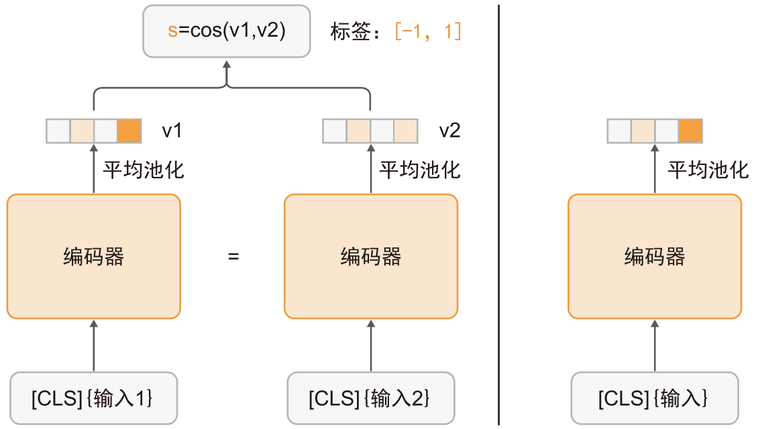


图 6.9　Sentence-BERT 的训练和测试方式示例

- 训练时，同一模型分别处理成对的文本，各自经过编码器与平均池化得到 $\boldsymbol{v}_1$、$\boldsymbol{v}_2$。
- 计算两个嵌入向量的余弦相似度 $s = \cos(\boldsymbol{v}_1, \boldsymbol{v}_2)$，作为损失计算的输入。
- 标签取 $[-1, 1]$：语义相关为正样本对，不相关为负样本对。
- 右侧为测试阶段：单条文本独立编码即可得到嵌入向量。
- 关键在于“共享编码器”：成对文本用同一套参数编码，相似度才有可比性。

图源 《大语言模型：训练、精调与对齐》图 6.9；**文献** Reimers & Gurevych, *Sentence-BERT*, EMNLP 2019

对比学习

为了训练模型生成更具判别性的嵌入向量，采用对比学习策略：让语义相关的文本对靠近，不相关的文本对远离。

正样本对

语义相关

如来自同一文档的相邻句子。目标：嵌入向量的相似度高。

负样本对

语义不相关

如来自不同文档的句子。目标：嵌入向量的相似度低。

成对文本

正/负样本对

→

同一模型编码

得到 $\boldsymbol{v}_1, \boldsymbol{v}_2$

→

计算相似度

$\cos(\boldsymbol{v}_1, \boldsymbol{v}_2)$

→

对比损失

反向传播更新参数

- 模型训练完成后，即可用于提取任意文本的嵌入向量。
- 以编码器 Transformer 设计的嵌入模型有 E5、BGE 和 GTE 等。

依据 《大语言模型：训练、精调与对齐》第 6.4.3 节；**文献** Wang et al., *Text Embeddings by Weakly-Supervised Contrastive Pre-training*, 2024

句向量模型的架构与评测

也有研究工作探索用基于解码器架构的 Transformer 模型生成文本嵌入向量。

- 一种典型做法：把输入序列末尾的 <EOS> 标记对应的模型输出向量作为整个序列的嵌入表示。
- Qwen-GTE、LLM-As-Vec、NV-Embd 等模型在此基础上增加了更多优化，以学习文本嵌入表示。

评测基准

MTEB

大规模文本嵌入基准（Massive Text Embedding Benchmark），涵盖表征、聚类、检索和分类等多种下游任务，以全面衡量嵌入模型能力。

嵌入模型的性能通常在该基准上评估。

架构	取向量方式	代表模型
编码器（BERT 类）	所有词元隐含层状态的平均池化	E5、BGE、GTE
解码器	序列末尾 <EOS> 对应的输出向量	Qwen-GTE、LLM-As-Vec、NV-Embd

6.5

智能体

让模型不只回答，还能感知环境、调用工具、依据观察调整行动。

本节内容

6.5.1 智能体介绍

6.5.2 ReAct

6.5.3 AI 智能体：深度研究

智能体的定义

在人工智能领域，智能体（Agent）指能够感知环境、自主采取行动以实现目标，并能通过学习提升自身表现的实体。

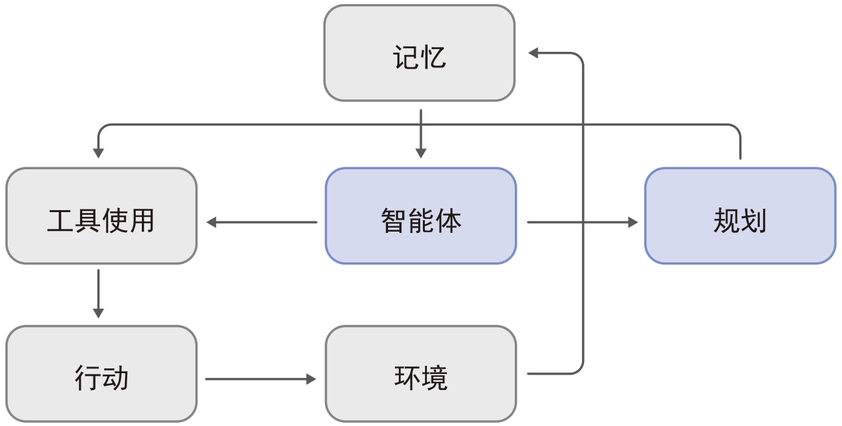


图 6.10　智能体及模块划分

- 随着大语言模型的发展，LLM 已成为构建智能体系统的核心“大脑”。
- 环境信息被编码为文本输入，LLM 依据提示词和自身能力进行推理与决策。
- 由此赋予智能体任务规划等高级能力：LLM 负责理解目标、拆解任务与选择动作，模块负责记忆与执行。

典型模块：规划（Planning）、记忆（Memory）、工具使用（Tool Use），以及与环境的行动闭环。

图源 《大语言模型：训练、精调与对齐》图 6.10

智能体的三个核心模块

规划模块

分解与反思

根据目标场景，将复杂任务分解为可管理的子目标，并结构化地高效执行。

还能对环境交互和过往行动进行自我反思与批评，从错误中学习并在后续步骤中改进。

记忆模块

长期与短期

存储形式多样：音频、图像、文本、信号等原始多模态数据，或经过处理的向量表示。

长期记忆依赖外部向量存储与高效检索，可长期保留和回忆近乎无限的信息；上下文学习能力可视为利用短期记忆（上下文窗口）进行学习。

工具使用

调用外部能力

调用外部 API 获取模型权重中缺失的信息（实时数据、专业信息源），或执行特定操作（如代码运行）。

操作终端设备时需在沙盒环境中运行，隔离潜在的不安全行为。

例 HuggingGPT 中，LLM（如 ChatGPT）充当协调中心，根据任务描述与 Hugging Face 平台上模型的功能描述，规划并调用合适的模型作为工具执行任务，最终整合结果。

依据 《大语言模型：训练、精调与对齐》第 6.5.1 节；**文献** Shen et al., *HuggingGPT*, NeurIPS 2023

智能体的任务执行

对于普遍的信息获取任务，智能体常借助工具检索外部信息（文档库、网络），获取相关上下文后再推理，从而提升准确性。此类方法主要包括 RAG 与基于网络搜索的内容生成。

需求
“做一个数据看板页面”

→

分解
查 API 文档、写代码

→

执行
生成 SQL、查询数据库

→

整合
返回结果，生成页面代码

- 要求智能体编写网页时，它能自主分解需求、查找 API 文档、编写代码、生成 SQL 查询数据库并返回结果，最终生成页面代码。
- 每一步的工具调用结果都作为下一步的输入，形成“决策—执行—观察”的闭环。
- 需要操作终端设备时，应在沙盒环境中运行以隔离潜在的不安全行为，保障系统安全。

智能体的三种范式

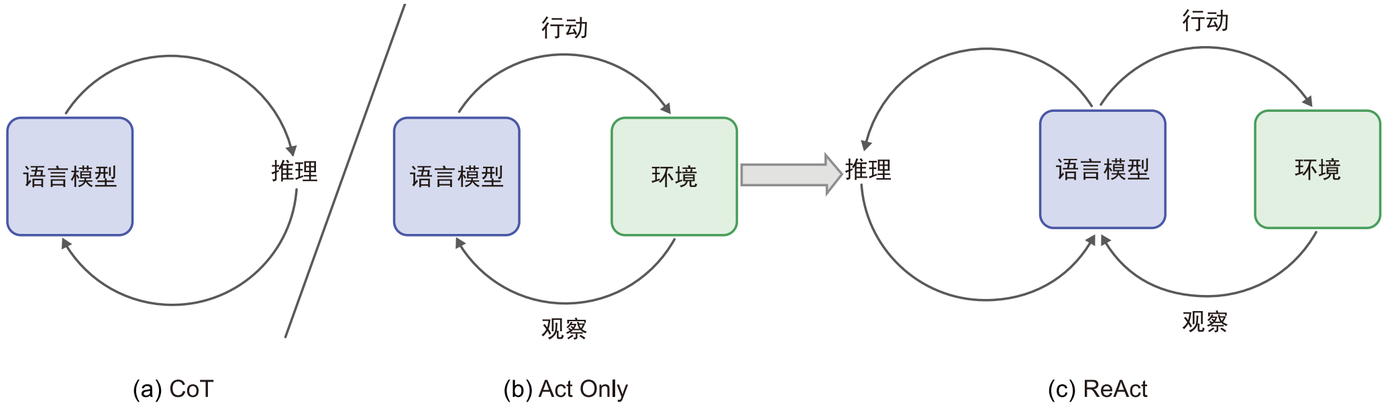


图 6.11　ReAct 框架：推理与行动的协同机制

(a) CoT 只在模型内部循环推理，不接触环境；(b) Act-Only 只行动不推理，缺少对结果的判断；(c) ReAct 让思考决定动作、观察结果再回到推理。自我反思是自主智能体的核心能力：审视过往决策与行动结果来修正错误。

图源 《大语言模型：训练、精调与对齐》图 6.11；**文献** Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023

ReAct 的循环流程

针对用户查询，智能体循环执行“推理-行动-观察”步骤，直至获得足够信息生成答案，或达到预设的最大尝试次数而终止。

推理

确定需要什么信息

→

行动

执行搜索等预设动作

→

观察

读取结果并评估有效性

→

判断

不足则重新推理，足够则回答

动作集合

可执行的动作限于预设集合，如 {思考，搜索，回答}。

每个行动步骤前通常包含“思考”过程，以提高决策质量。

```
思考：...（推理步骤）
行动：...（执行动作）
观察：...（获取反馈）
...（循环进行）
```

若结果不满足需求，则重新推理并调整行动策略。

ReAct 的示例

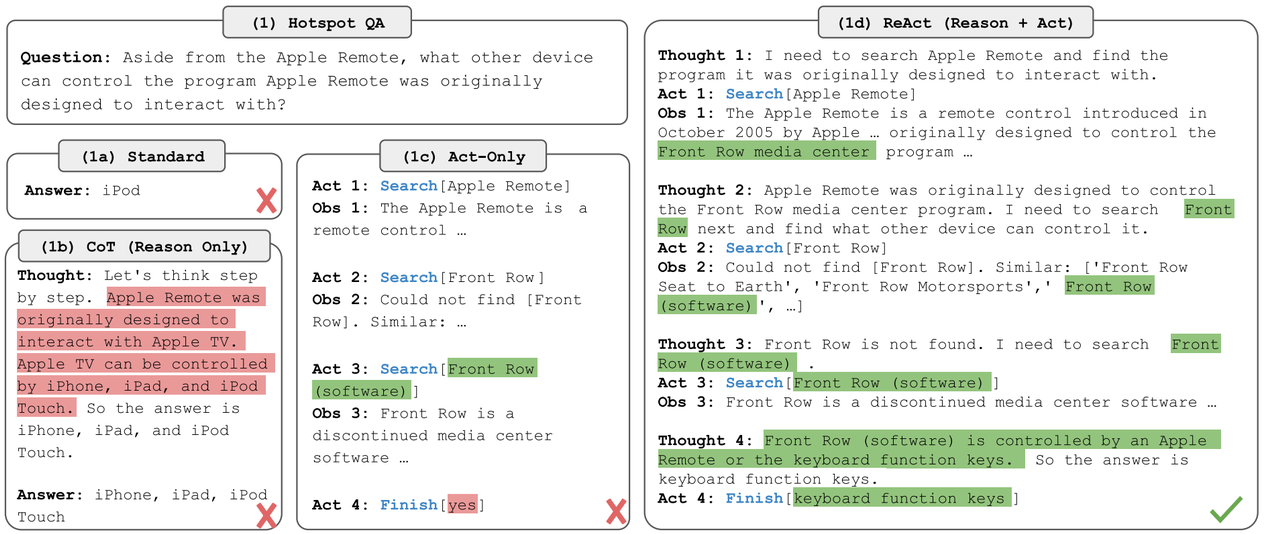


图 6.12　ReAct 示例

- 以 HotpotQA 问答任务为例，查询“苹果公司开发的远程控制程序”。
- ReAct 首先推理确定需要搜索的关键词，执行搜索动作后从返回内容中提取相关信息。
- 经过多轮“推理—搜索—观察”循环，直到获得充分证据才输出最终答案。
- 左侧为多轮轨迹：每次行动都留痕，可追溯答案来自哪一次检索。

图源 《大语言模型：训练、精调与对齐》图 6.12；**文献** Yao et al., *ReAct*, ICLR 2023；Yang et al., *HotpotQA*, EMNLP 2018

智能体的优势与局限

优势

通过实时检索外部互联网信息，获取最新内容并注明来源。

有效减少模型仅依赖内部知识产生的幻觉，显著提升回答的可信度与可验证性。

行动轨迹使答案有据可查。

局限

高度依赖预设的有限动作集合与规则流程。

检索结果的质量直接影响输出准确性；若检索失败或返回错误上下文，反而可能生成错误答案。

模型仍需具备足够的辨别能力来评估信息可靠性。

研究方向 当前 ReAct 类框架需在预设规则下运行；如何实现开放环境中的自主决策、复杂动作与工具使用，是重要的研究方向。强化学习技术可通过经验学习提升模型在复杂场景中的自适应推理与决策能力（详见第 7、8 章）。

Deep Research

OpenAI 的深度研究（Deep Research）是一个 AI 智能体，能自主地在互联网上进行多步骤研究，处理复杂任务，将数小时的人工工作缩短到数分钟。

角色

研究分析师

查找、分析并综合数百个在线来源，生成全面的报告。

底座

o3 模型

专门针对网络浏览与数据分析优化，利用推理能力解释海量在线文本、图像与 PDF。

能力

调整方法

根据获取的新信息调整研究方法，而非固守初始计划。

产出

带引用的报告

使用 Python 工具绘图并迭代，嵌入图表与网页图片，引用来源中的特定句子或段落。

这种知识综合能力是通向通用人工智能的重要一步：它体现了合成知识的关键技能，而合成知识正是创造新科学发现的前提。

Deep Research 的特点

维度	传统 Agent	Deep Research
任务范围	多用于简单任务	各种领域中复杂的浏览与推理任务
训练方式	依赖预设规则与流程	端到端强化学习训练
执行路径	按固定步骤执行	自主规划并执行多步骤路径
调整能力	有限	必要时根据实时信息回溯与调整
输入与产出	以文本为主	可浏览用户上传文件，用 Python 绘图并嵌入响应，标注引用

- 通过端到端强化学习，模型学会了如何规划和执行多步骤路径来查找所需数据。
- 这些训练使它在多项针对实际问题的公开评估中达到新的高度。

依据 《大语言模型：训练、精调与对齐》第 6.5.3 节

6.6

自指令数据合成

人工标注指令数据成本高；用模型自己生成、自己过滤，再拿来做指令微调。

本节内容

6.6.1 指令跟随能力

6.6.2 自指令数据流程

6.6.3 数据标注

指令跟随能力

简单的对话即可获取信息；而在处理复杂且具体的任务要求时，模型需要能够理解用户意图，并生成能有效解决问题的方案。具备这种良好回答能力的模型，拥有指令跟随能力。

示例任务

问题： 总结近三年大语言模型领域中的5个关键技术突破：
（1）要求内容包括：技术问题、解决方法、解决时间和具体参考出处
（2）要对比新技术较旧技术的优劣
（3）所有结果以表格形式展现
（4）输出 Latex 格式结果
回答：

- 这个任务同时约束了内容范围、比较维度、呈现形式与输出格式。
- 基础预训练模型通常缺乏稳定的对话能力；直接输入上述问题，模型未必按格式作答。
- 更直接的方法是收集通用的指令任务数据做微调，使模型具备指令跟随能力；此后用户可直接根据问题与模型交互。

指令数据的格式

指令数据是包含“指令 + 输入”和“输出”的成对数据，覆盖广泛的专业领域与日常使用场景。基于指令数据做的微调称为指令微调，通过有监督学习把遵循指令的能力注入模型内部。

示例 1

指令：写一个名人名言。

输入：与意志有关的主题。

输出：生活就像海洋，只有意志坚强的人，才能到达彼岸。——马克思

示例 2

指令：中译英

输入：天安门广场

输出：Tian'anmen Square

示例 3

指令：什么是人工智能

输入：（空）

输出：人工智能是指模拟和扩展人类智能的技术和系统，通过学习、推理、感知和决策来完成任务。

输入是可选字段：有的任务只需指令与输出，如示例 3。

对话模板

区别于预训练数据，指令数据通常有特定的格式要求，训练前需处理成统一的字符串形式；指令微调后的模型也需遵循给定的对话模板。

模板

```
用户：{指令} {输入}
AI助手：{输出}
```

同一模板套用到所有样本，模型才能学到“用户说什么、助手答什么”的稳定分工。

格式化后

```
用户：写一个名人名言。与意志有关的主题。
AI助手：生活就像海洋，只有意志坚强
的人，才能到达彼岸。—马克思

用户：中译英。 天安门广场
AI助手：Tian'anmen Square

用户：什么是人工智能?
AI助手：人工智能是指模拟和扩展人类
智能的技术和系统，通过学习、推理、
感知和决策来完成任务。
```

模板把“指令+输入”合并为用户侧内容，输出作为助手侧内容，形成一条完整对话。

依据 《大语言模型：训练、精调与对齐》第 6.6.1 节

Self-Instruct 方法

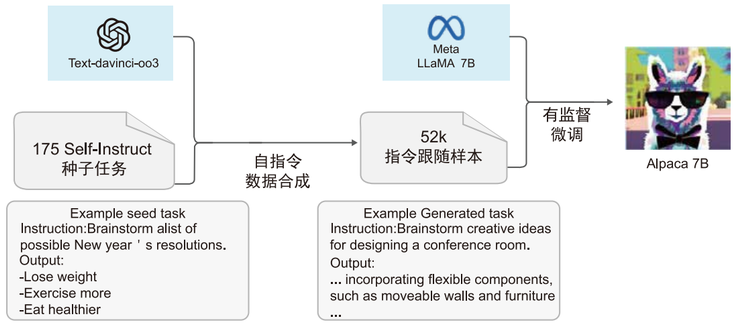


图 6.13　自指令数据合成和 Alpaca-7B 模型训练

- 人工收集指令数据成本高，尤其在获取高质量数据与人工标注方面。
- 自指令方案提出通过提示词工程生成大规模指令数据：以种子指令为基础，经生成、分类、补全和过滤的迭代流程合成新指令。
- 合成过程强调指令多样性，避免模式单一。

> **规模**　利用 ChatGPT-3.5 生成 5.2 万条指令数据构建 Alpaca 数据集；用该数据集微调 Llama 7B，得到的 Alpaca 7B 展现出优异的指令跟随能力。

图源 《大语言模型：训练、精调与对齐》图 6.13；**文献** Wang et al., *Self-Instruct*, ACL 2023；Taori et al., *Alpaca*, 2023

Self-Instruct 的种子指令


write
give
find
create
make
describe
design
generate
classify
have
explain
write
function
letter
essay
paragraph
example
list
set
advice
word
number
sentence
way
program
list
algorithm
function
list
goes
situation
person
system
algorithm
algorithm
list
number
sentence
sentence
sentence
list
list
difference
concept
story
sentence
sentence
number
sentence
sentence
sentence

Self-Instruct 的迭代步骤

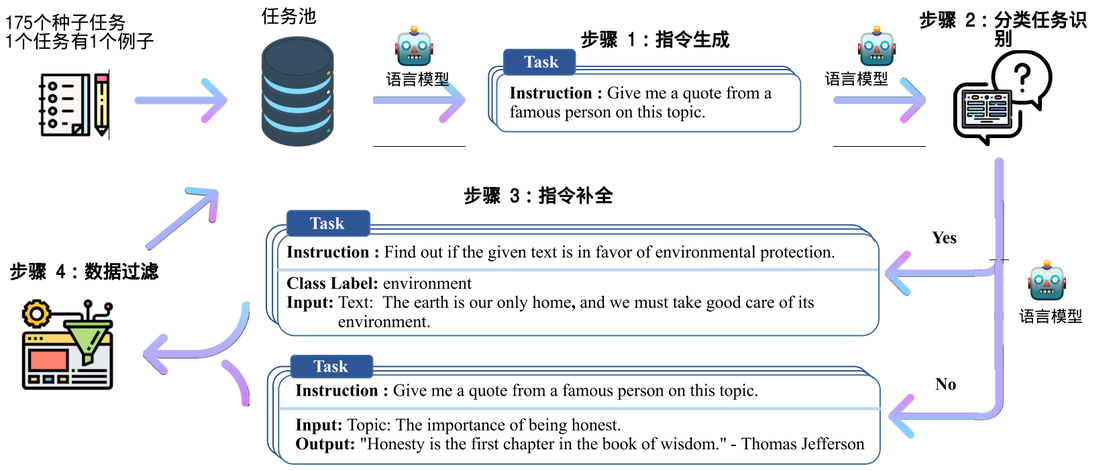


图 6.15　自指令数据合成的流程图

① 指令生成

从种子池与生成池采样，提示模型写新指令

② 指令分类

分类任务 / 生成任务

③ 指令补全

补全输入与输出

④ 数据过滤

按重叠度与质量阈值筛选

合格样本加入指令池，循环执行 ①～④ 直至达到预设数据量。

图源 《大语言模型：训练、精调与对齐》图 6.15；**文献** Wang et al., *Self-Instruct*, ACL 2023

Self-Instruct 步骤 ①

```
根据以下指令生成新指令：
指令1：{种子指令1}
...
指令6：{种子指令6}
指令7：{生成指令1}
指令8：{生成指令2}
指令9：

（含 8 个样本的提示模板）
```

- 定义包含 8 个样本的提示模板：6 个样本从种子指令池随机采样，2 个样本从生成指令池采样（若存在 $\geqslant 2$ 个样本）。
- 将模板输入语言模型（如 ChatGPT）生成新指令。
- 使用 Self-BLEU 指标评估生成指令与样本的相似度，舍弃高相似度指令，以保证数据多样性。

两类样本池混采：种子保证任务广度，生成池让后续指令延续已有风格并逐步扩展。

Self-Instruct 步骤 ②

将指令分为两类，决定后续如何补全。

类别一

分类任务

输出为预定义类别。

类别二

生成任务

需要自由文本输出。

分类方法常规可用已训练的专用分类模型；Self-Instruct 使用少样本学习进行生成式分类。

分类少样本提示词

```
判断指令是否为分类任务
（是:1，否:0):
{指令1} 分类: 1
{指令2} 分类: 0
...
{指令} 分类:
```

要点 用同一个生成模型做分类，避免额外训练专用分类器；判断结果只影响下一步补全的方式。

依据 《大语言模型：训练、精调与对齐》第 6.6.2 节

Self-Instruct 步骤 ③ 与 ④

步骤 ③ 指令补全

```
根据示例，补全输入和输出内容：
{指令1}  {输入1}  {输出1}
{指令2}  {输入2}  {输出2}
...
{指令}
```

根据指令类型，补全少样本示例、输入与输出内容。

步骤 ④ 数据过滤

质量与多样性双重筛选

评估生成数据质量，使用 ROUGE 或 BLEU 指标检测与指令池的重叠度，通过质量阈值过滤低质量样本。

合格样本加入指令池，用于后续生成；循环执行 ①～④ 直至达到预设数据量。



补全决定样本是否可用，过滤决定样本是否入池：前者按指令类型补出输入与输出，后者用重叠度阈值剔除与已有指令过于相似的样本。

模型自动标注数据

在自指令数据构造的指令分类任务中，可采用少样本学习方法对新增样本进行标注。除分类外，提示词工程还可用于标注其他任务类型。

任务一

分类标注

在提示中给出少量带标签的示例，让模型判断新样本的类别。

任务二

实体识别标注

在提示中提供包含输入和对应标签的示例，要求模型输出实体标签。

建议将示例标签设计为 JSON 等结构化文本格式，便于后续自动化处理。

任务三

偏好标注

对同一问题生成多个回答，让模型对这些回答的质量排序或做偏好标注。

偏好数据是训练奖励模型的关键，常用于强化学习人类反馈流程（详见第 8 章）。

适用范围 模型标注主要适用于特定且相对简单的任务；为评估标注准确性，可将其结果与人工标注进行对比。

提高标注质量的策略

策略一

多次采样与一致性判断

通过模型概率采样进行多次标注，依据结果的一致性（Consistency）筛选可靠标注。

多次结果一致，说明模型对该样本较有把握。

策略二

多模型交叉验证

使用多个高性能语言模型进行独立标注，对一致性较低的数据进行筛选。

不同模型的错误模式不同，交叉比对能暴露分歧样本。

策略三

人机协作

将上述方法识别出的低一致性样本交由人工审核标注。

把人工投入集中在最不确定的样本上，兼顾成本与质量。

模型标注

全量样本

→

一致性筛选

多次采样 + 多模型

→

人工审核

只处理低一致性样本

依据 《大语言模型：训练、精调与对齐》第 6.6.3 节

6.7 评估

模型能力越来越通用，如何量化它、比较它，成为独立的问题。

本节内容

6.7.1 模型评估
6.7.2 基准测评
6.7.3 胜率测评
6.7.4 竞技场排位
6.7.5 基准测评方法

模型评估的三个问题

随着语言模型能力日益通用化，量化其性能变得至关重要。

问题一

能力边界在哪

在特定数据集上评估其基础性能，明确模型能做什么、不能做什么。

问题二

与人类差多少

测评语言模型与人类能力的差异。

问题三

模型之间孰优

引入胜率与排位模型（ELO 排位）来反映模型的相对性能。

预训练模型

测评主要反映其在预训练阶段学习到的知识量或广度。

指令微调模型

测评侧重于在具体任务上的表现，关注是否按指令完成任务。

两类模型的测评侧重点不同，不能用同一套指标简单比较。

3H 原则

Helpful

有用性

模型输出满足用户需求的程度，可进一步细分为多个维度。

Honest

诚实性

不虚构、不夸大，对不确定的内容如实说明。

Harmless

安全性

不产生有害内容，能抵御不当请求。

Anthropic 提出评估指令微调模型的 3H 原则：有用性、安全性与诚实性。

测评形式	判定方式
问答、填空	设定特定输出格式，用正则表达式与参考答案匹配
选择题	比较各选项标识符的预测概率或困惑度
开放式生成	把生成的答案输入另一个语言模型做一致性判断
交互式任务	在相应运行环境中执行代码或调用工具，基于结果评估（RAG、Agent、代码生成）

依据 《大语言模型：训练、精调与对齐》第 6.7.1 节；**文献** Askell et al., *A General Language Assistant as a Laboratory for Alignment*, 2021

评估方法的演进

生成答案
不同模型回答同一测评集

→

两两比较
人工评审判断谁更好

→

计算胜率
统计各模型的胜负

→

全局评分
Bradley-Terry 模型拟合

- 比较多个模型时，常见做法是让不同模型在测评集问题上生成答案，再由人工评审判断哪个输出更优，从而计算各模型的胜率。
- 在多个模型比较的场景中，可基于两两之间的胜率关系，利用 Bradley-Terry 模型计算全局评分。
- 增加评审员数量可以提高评判的一致性；使用语言模型作为裁判（LLM-As-a-Judge）则能提高测评效率、降低成本。
- 尽管存在标准的基准测试集，测试集污染问题可能导致部分模型“作弊”，检测作弊行为是测评研究中的重要课题。

依据 《大语言模型：训练、精调与对齐》第 6.7.1 节；**文献** Ford, *Solution of a Ranking Problem from Binary Comparisons*, 1957

评估的能力维度

对通用语言模型的基准测评旨在明确性能边界、适用场景、潜在改进方向，并评估其可靠性与安全性。

能力	评估内容	代表基准
多语言能力	翻译、文本生成、阅读理解等跨语言任务	XTREME、MGSM
知识能力	百科与专业知识的准确性、广度、时效性	MMLU、CMMLU、C-Eval
推理能力	文字推理、归纳、演绎与假设验证	GSM8K、ARC、GPQA
代码能力	代码生成、错误修复、算法实现；需实际运行	HumanEval、MBPP、LiveCodeBench
数学能力	基础运算、代数、几何及更高阶领域	MATH、AIME
安全性	面对恶意、偏见或诱导性内容时的响应	StrongReject、XSTest
工具使用能力	调用 API、数据库、计算器完成任务	ToolBench、SWEBench

除上述通用维度外，还需关注面向特定任务的专项评估，如长文本处理能力、信息检索能力以及指令遵循能力。

胜率测试

胜率测试（Winrate Test）通过直接比较不同语言模型生成结果的优劣，评估它们在特定任务上的相对表现。

适用

主观性强的任务

同一模型的不同配置或不同尺寸之间的比较；对话生成、创意写作、开放式问答等存在多种解法的任务。

能直观展示模型改进的效果或竞争模型之间的性能差异。

对照

与基准测评的差别

基准测评依赖数值指标（如准确率）；胜率测试直接对比生成结果本身。

更侧重反映人类对结果的偏好（流畅性、相关性），能更好地模拟用户选择模型输出的真实过程。

- 经过人类反馈强化学习训练的语言模型，其输出偏好已与人类对齐，因此可以用模型本身代替人工进行胜率评判。
- AlpacaEval 采用动态更新的数据集作为测评基准，支持更广泛的评估，并有助于避免因使用公开固定测试集导致的数据污染（如模型过拟合）。

ELO 评分

在大语言模型领域，ELO 评分系统常被用于对多个模型性能进行排位。它最初用于棋类比赛，根据参赛者的胜负记录动态调整评分。

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}} \tag{6-1}$$

$$E_B = \frac{1}{1 + 10^{(R_A - R_B)/400}} \tag{6-2}$$

$$R'_A = R_A + K\left(S_A - E_A\right) \tag{6-3}$$

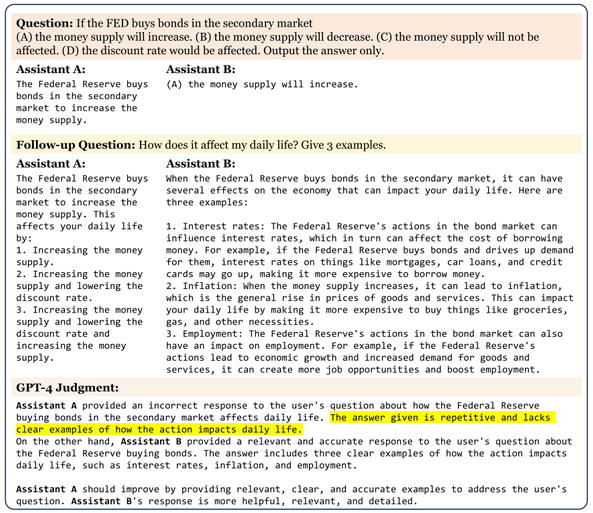
$$R'_B = R_B + K\left(S_B - E_B\right) \tag{6-4}$$

E 为预期得分，R 为当前评分，K 为单场最大调整幅度，S 为实际结果：胜 = 1，平 = 0.5，负 = 0。

- 初始评分：每位参与者（模型）初始评分通常设为 400 分。
- 对战结果：给定一个问题，由两个不同的模型生成回答，再由人类或另一个模型（评判者）判别优劣。
- 评分更新：胜者评分增加，败者减少；若为平局，双方变化较小。
- 常数 K 通常在 10 至 40 之间，代表单场比赛评分的最大调整幅度。

依据 《大语言模型：训练、精调与对齐》第 6.7.4 节（式 (6-1)–(6-4)）

MT-Bench



Question: If the FED buys bonds in the secondary market
(A) the money supply will increase. (B) the money supply will decrease. (C) the money supply will not be affected. (D) the discount rate would be affected. Output the answer only.

Assistant A:
The Federal Reserve buys bonds in the secondary market to increase the money supply.

Assistant B:
(A) the money supply will increase.

Follow-up Question: How does it affect my daily life? Give 3 examples.

Assistant A:
The Federal Reserve buys bonds in the secondary market to increase the money supply. This affects your daily life by:
1. Increasing the money supply.
2. Increasing the money supply and lowering the discount rate.
3. Increasing the money supply and lowering the discount rate and increasing the money supply.

Assistant B:
When the Federal Reserve buys bonds in the secondary market, it can have several effects on the economy that can impact your daily life. Here are three examples:

1. Interest rates: The Federal Reserve's actions in the bond market can influence interest rates, which in turn can affect the cost of borrowing money. For example, if the Federal Reserve buys bonds and drives up demand for them, interest rates on things like mortgages, car loans, and credit cards may go up, making it more expensive to borrow money.
2. Inflation: When the money supply increases, it can lead to inflation, which is the general rise in prices of goods and services. This can impact your daily life by making it more expensive to buy things like groceries, gas, and other necessities.
3. Employment: The Federal Reserve's actions in the bond market can also have an impact on employment. For example, if the Federal Reserve's actions lead to economic growth and increased demand for goods and services, it can create more job opportunities and boost employment.

GPT-4 Judgment:

Assistant A provided an incorrect response to the user's question about how the Federal Reserve buying bonds in the secondary market affects daily life. The answer given is repetitive and lacks clear examples of how the action impacts daily life.
On the other hand, **Assistant B** provided a relevant and accurate response to the user's question about the Federal Reserve buying bonds. The answer includes three clear examples of how the action impacts daily life, such as interest rates, inflation, and employment.

Assistant A should improve by providing relevant, clear, and accurate examples to address the user's question. **Assistant B**'s response is more helpful, relevant, and detailed.

图 6.16　语言模型作为裁判

```
问题：{问题}
智能体A：{回答A}
智能体B：{回答B}
请根据问题，分析回答A和回答B的好坏，
并在最后给出结论哪个回答更好。
GPT-4 评判：
```

- MT-Bench 提供在线数据集和评估框架，按上述模板呈现两个模型的回答，由评判模型判别优劣。
- 需选择与人类评估一致性较高的语言模型（如 GPT-4）作为自动评判员。

图中 GPT-4 的评判既给出结论，也说明了理由：A 回答了不同的问题，B 更贴题且举例具体。

图源 《大语言模型：训练、精调与对齐》图 6.16；**文献** Gu et al., *A Survey on LLM-as-a-Judge*, 2025

在线竞技场评估

在线聊天机器人竞技场支持真实用户动态地进行模型两两对比测评，并实时更新 ELO 排名。

用户提问

真实、开放的问题

→

匿名对战

两个模型各自作答

→

用户投票

判断哪个回答更好

→

实时更新

累计胜负，刷新 ELO 排名

优点

真实偏好

题目来自真实用户，投票反映真实偏好，贴近实际使用场景。

优点

动态更新

排名随新对战实时变化，可观察模型迭代带来的位次变化。

注意

需结合基准

主观偏好无法覆盖客观能力，排位应与基准测评结合解读。

依据 《大语言模型：训练、精调与对齐》第 6.7.4 节；**文献** Chiang et al., *Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference*, 2024

MMLU 基准

MMLU（Massive Multitask Language Understanding）是评估跨领域语言理解能力的基准数据集。

- 涵盖广泛主题，问题以选择题形式呈现，利于评判。
- 旨在全面测试模型的知识广度与理解能力。
- 测评时可采用少样本学习方式，在问题前拼接少量示例作为上下文。

问题格式

```
问题：{问题内容}，选项为
 (A){内容A}
 (B){内容B}
 (C){内容C}
 (D){内容D}
```

两种打分方式

按概率选择：模型预测下一个词元是“A”“B”“C”或“D”的概率，取最高者。

按困惑度选择：把完整问题与选项拼接后计算 PPL，或分别计算各选项标识符的负对数似然，取困惑度最低或概率最高的选项。

自由格式问答的评估

对于包含客观答案的自由格式问答（如 MATH），答案标签通常以特定格式给出，例如 `\boxed{}`。

生成 → **抽取** → **比对** → **判定**

- **生成**：模型输出完整解答
- **抽取**：取 `\boxed{}` 内的字符串
- **比对**：与标签字符串匹配
- **判定**：精确匹配 / 等价性判断

例子

从 `\boxed{1/2}` 中提取出 `1/2`，与标签字符串 `0.5` 比对。

若直接做字符串匹配，“1/2”与“0.5”不相等，会判定为错误。

改进

可通过预设的规则进行数值判别，或把两者输入 LLM 判断其数学意义是否等价，以此评估正确性。

自由格式问答常见于数学题目等场景。

由于模型生成的中间推理步骤通常难以有效评估，当前提升推理能力的重要手段之一是通过规则奖励来激发模型的推理能力（详见第 8 章）。

依据 《大语言模型：训练、精调与对齐》第 6.7.5 节；**文献** Hendrycks et al., *Measuring Mathematical Problem Solving With the MATH Dataset*, 2021

6.8 小结

本节内容

本章回顾

应用版图与展望

提示词工程是外部优化；它一头应对模型局限，一头拓展能力边界。

本章小结

语言模型解决问题通常有两种路径：微调模型参数，或调整输入内容以引导正确输出。后者即提示词工程，其基础是上下文学习范式。

内部优化

调整参数 θ：预训练、有监督微调、指令微调、强化学习。

能力写进权重，推理时不再依赖长提示。

方向一

应对模型局限

针对知识缺失或概率本质产生的幻觉，可通过内部方法（如自一致性）或引入外部信息来缓解；检索增强生成是后者的典型应用，通过固定流程检索信息补充事实上下文。

外部优化

调整输入 $\boldsymbol{x}$：指令、示例、思维链、检索上下文、工具调用。

即改即用，是本章的主线。

方向二

拓展能力边界

超越单一语言交互，构建智能体框架，使 LLM 主动与外部环境交互（查询信息、使用工具）以解决更复杂问题；并通过强化学习优化自主工具使用与交互的策略。

提示词工程的关键突破

关键突破在于：以 GPT-3 为代表的大语言模型在无微调情况下展现了强大的通用任务泛化能力，且不依赖特定训练数据分布。这为 LLM 通向通用人工智能奠定了重要实践基础。

数据合成

Self-Instruct，减少人工标注依赖

数据标注

分类、实体、偏好标注

模型评估

对战评测、基准测试、排行榜

代码生成

以提示组织工程化目标

- 提示词工程的应用远超特定任务优化，已广泛应用于数据合成、数据标注、模型评估、代码生成等多个领域。
- 核心价值在于通过灵活、结构化的提示组织实现工程化目标。
- 主流的 LLM 性能评估方法包括对战评测、基准测试和排行榜，可按判别标准（客观/主观）分类。
- LLM 的通用能力是核心优势，其各项子能力（理解、推理、生成等）相互关联、协同。

凭借通用性与泛化能力，LLM 已成为跨领域应用的核心引擎：在多模态系统中充当关键桥梁，并在智能体场景中不断突破应用边界。

6.9

附录：In-Context learning 理论*

上下文学习为什么有效？三种理论视角给出互补的解释。

本节内容

6.9.1 贝叶斯视角
6.9.2 梯度下降视角
6.9.3 信息论视角

上下文学习的贝叶斯解释

在贝叶斯框架下，上下文学习可被形式化为隐式贝叶斯推断过程。

核心假设

语言模型在预训练阶段学习了一个由潜在概念混合组成的生成分布；测试阶段通过提示中的示例推断出共享的潜在概念，进而完成预测。

- 有效性依赖预训练分布中隐含的长程连贯性：预训练文档通过潜在概念（主题、语法结构或任务范式）生成连贯序列。
- 模型在推断提示时，通过隐变量后验集中到特定概念上。

概念

潜在概念 $\theta \in \Theta$ 决定文档的主题与结构。

数据

文档即由 θ 对应的生成过程产出的观测序列。

推断

提示示例是观测，后验集中即“认出”了任务。

依据 《大语言模型：训练、精调与对齐》第 6.9.1 节；**文献** Xie et al., 2021；Wies et al., 2023

隐变量模型与预训练分布

假设预训练数据服从隐马尔可夫模型（HMM）混合分布。每个文档的生成过程包含两步。

- 从先验分布 $p(\theta)$ 采样潜在概念 $\theta \in \Theta$。
- 根据 θ 对应的 HMM 转移矩阵生成序列 $o_1, o_2, \cdots, o_T$。

例如维基百科传记类文档中，潜在概念可能对应人物身份、时间线等结构化信息，模型需通过跨句推断维持概念一致性。

$$p\left(o_1, o_2, \cdots, o_T\right) = \int_{\theta \in \Theta} p\left(o_1, o_2, \cdots, o_T \mid \theta\right) p(\theta)\, \mathrm{d}\theta$$

预训练目标为最大化边缘似然：对潜在概念积分，模型必须推断出 θ 才能生成连贯的后续词元。

依据 《大语言模型：训练、精调与对齐》第 6.9.1 节

提示分布与后验推断

测试阶段的提示由 n 个独立示例拼接而成，形式上与自然语言文档存在分布偏移。

$$S_n = \left[x_1, y_1, o^{\text{delim}}, x_2, y_2, o^{\text{delim}}, \cdots, x_n, y_n, o^{\text{delim}}\right]$$

$$p\left(y \mid S_n, x_{\text{test}}\right) = \int_{\theta} p\left(y \mid x_{\text{test}}, \theta\right) p\left(\theta \mid S_n\right) \mathrm{d}\theta$$

给定提示 S_n 与测试输入 x_{test}，预测输出的后验分布。数学本质是贝叶斯模型平均。

分布偏移

预训练文档可能是“姓名→国籍→职业”，提示却是“姓名→国籍→分隔符→姓名→国籍”，产生低概率转移（如“德国→甘地”）。

仍然有效

模型仍能通过隐变量后验的集中性实现上下文学习：当 $p(\theta \mid S_n)$ 集中到提示的真实概念 θ^* 时，通过边缘化选择 θ^* 完成预测。

可区分性条件

后验能否集中到正确概念，取决于示例信号是否强于分布偏移带来的误差。

$$\sum_{j=1}^{k} \mathbb{E}_{O\sim p_{\text{prompt}}}\left[\mathrm{KL}\left(p^{j}_{\text{prompt}} \,\|\, p^{j}_{\theta}\right)\right] > \epsilon^{\theta}_{\text{start}} + \epsilon^{\theta}_{\text{delim}}$$

对任意 $\theta \neq \theta^*$，提示示例在 θ^* 下的平均对数似然比应显著高于其他概念。

- $\epsilon^{\theta}_{\text{start}}$ 与 $\epsilon^{\theta}_{\text{delim}}$ 分别表示提示起始分布和分隔符引入的分布偏移误差。
- 当每个示例的信号强度（KL 散度）超过分布偏移误差时，后验概率随示例数量 n 指数收敛到 θ^*。
- 这解释了一个经验事实：示例的“信息量”比数量更关键；格式噪声大时，多加示例的边际收益会下降。

渐近最优性

$$\arg\max_{y} p\left(y \mid S_n, x_{\text{test}}\right) \to \arg\max_{y} p_{\text{prompt}}\left(y \mid x_{\text{test}}\right), \qquad n \to \infty$$

在可区分性条件下，上下文预测器随示例数量趋于贝叶斯最优分类器。证明依赖改进的 Bernstein-von Mises 定理，允许在非独立同分布和非平稳序列下仍保证后验集中性。

$$\lim_{n\to\infty} L_{0-1}\left(f_n\right) \leqslant \inf_{f} L_{0-1}(f) + g^{-1}\left(O\left(\frac{\gamma_{\theta^*} \sup_{\theta\in\mathcal{B}}\left(\epsilon_{\text{start}}^{\theta} + \epsilon_{\text{delim}}^{\theta}\right)}{k-1}\right)\right)$$

当潜在概念连续或不可区分时，误差仍随示例长度 k 衰减；γ_{θ^*} 为 Fisher 信息矩阵的条件数，$g(\delta)$ 为校准函数。

该结果表明：输入序列的语义信息（而不仅是输入-输出映射）对上下文学习至关重要——长示例提供更多关于潜在概念的证据，降低后验推断的不确定性。

任务识别

在隐式贝叶斯视角下，上下文学习的本质是识别预训练中已存在的潜在任务，而非从零学习新任务。

证据一

标签无关性

即使提示中的标签被随机替换，模型仍能通过输入分布识别任务。

证据二

先验覆盖性

预训练混合分布需覆盖下游任务的潜在概念，且先验 $p(\theta)$ 有足够支撑（$p(\theta^*) \geqslant c_3 > 0$）。

证据三

样本复杂度

在多项式数量的示例 k 下，后验概率比 $p(\theta \mid S_n)/p(\theta^* \mid S_n)$ 以指数速率收敛，保证任务识别误差可控。

含义 若预训练没见过某类任务，示例再多也难以“认出”它；反之，只要概念在预训练分布中有支撑，少量示例即可完成识别。这解释了 ICL 的强项与盲区。

注意力与梯度下降的对应

Transformer 中的注意力机制在数学形式上与梯度下降存在深刻联系。考虑线性层的参数更新。

$$(\boldsymbol{W}_0 + \Delta\boldsymbol{W})\,\boldsymbol{x}, \qquad \Delta\boldsymbol{W} = \sum_i \boldsymbol{e}_i \otimes \boldsymbol{x}_i'$$

参数更新后的线性变换；$\Delta\boldsymbol{W}$ 由历史输入向量 $\boldsymbol{x}_i'$ 与误差信号 $\boldsymbol{e}_i$ 的外积累加构成。

$$\mathcal{F}(\boldsymbol{x}) = \boldsymbol{W}_0\boldsymbol{x} + \sum_i \boldsymbol{e}_i \left(\boldsymbol{x}_i'^T \boldsymbol{x}\right) = \boldsymbol{W}_0\boldsymbol{x} + \mathrm{LinearAttn}\left(\boldsymbol{E}, \boldsymbol{X}', \boldsymbol{x}\right)$$

梯度更新机制与线性注意力操作在数学上等价。

两者都把“输入向量 × 误差信号”的外积累加进参数：梯度下降在反向传播中累加，注意力在前向计算中按输入顺序累加。

线性注意力下的隐式参数更新

$$\tilde{\mathcal{F}}_{\mathrm{ICL}}(q) = \boldsymbol{W}_{\mathrm{ZSL}} q + \sum_i \left(\boldsymbol{W}_v \boldsymbol{x}'_i\right) \left(\boldsymbol{W}_k \boldsymbol{x}'_i\right)^T q$$

将标准注意力中的 Softmax 归一化去除，得到松弛的线性注意力形式；$\boldsymbol{W}_{\mathrm{ZSL}}$ 对应零样本学习的初始参数。

- 第二项可视为通过注意力机制实现的隐式参数更新 $\Delta \boldsymbol{W}_{\mathrm{ICL}}$。
- 这揭示了注意力机制本质上执行着元梯度操作：模型将示范样本的键值对 $\left(\boldsymbol{W}_k \boldsymbol{x}'_i, \boldsymbol{W}_v \boldsymbol{x}'_i\right)$ 视为元梯度信息。
- 通过注意力权重与当前查询的交互，完成参数空间的调整——前向计算即完成了一次“更新”。

依据 《大语言模型：训练、精调与对齐》第 6.9.2 节

上下文学习与显式微调的对应

对应一

梯度的隐式与显式实现

ICL 通过前向计算生成元梯度，注意力把这些梯度作用于初始参数；显式微调则用反向传播得到 $\Delta \boldsymbol{W}_{\mathrm{FT}}$。

两者都满足 $\Delta \boldsymbol{W} = \sum(\text{梯度分量} \otimes \text{输入分量})$，区别仅在梯度来源。

对应二

信息源与顺序一致

两种方法共享相同的训练样本信息，且都保持样本呈现的因果顺序。

ICL 中后续示例无法影响前面示例的处理；显式微调采用单次遍历，保证参数更新顺序的因果性。

对应三

注意力的定向调整

ICL 通过扩展键值对引入新信息，受限微调仅调整键值投影矩阵，二者对注意力的改造具有等效性。

$\Delta \boldsymbol{W}_{\mathrm{ICL}} = \boldsymbol{W}_v \boldsymbol{X}'(\boldsymbol{W}_k \boldsymbol{X}')^T$ 对 $\Delta \boldsymbol{W}_{\mathrm{FT}} = \Delta \boldsymbol{W}_v \boldsymbol{X}(\boldsymbol{W}_k + \Delta \boldsymbol{W}_k)^T$。

结论 这种结构对称性表明：ICL 可视为在参数空间之外构建的隐式微调过程，通过注意力机制动态调整模型行为。

动量机制

基于梯度下降的类比，可将动量机制引入注意力过程，形成改进方案。

$$\Theta_t = \Theta_{t-1} - \gamma \sum_{i=1}^{t-1} \eta^{t-i} \nabla f_{\Theta_i}$$

传统动量梯度下降：当前更新累积历史梯度的指数加权和。

$$\mathrm{MoAttn}\left(\boldsymbol{V}, \boldsymbol{K}, \boldsymbol{q}_t\right) = \mathrm{Attn}\left(\boldsymbol{V}, \boldsymbol{K}, \boldsymbol{q}_t\right) + \sum_{i=1}^{t-1} \eta^{t-i} \boldsymbol{v}_i$$

对应到注意力机制：第二项实现历史注意力值的指数移动平均。

- 这种设计强化了注意力对近期信息的偏好，与语言建模中的近期偏好（recency bias）理论相契合。
- 动量机制的引入验证了对偶理论的实践价值，也为模型优化提供了新的设计维度。

世界模型

在形式语言学框架下，构建由有限对象集合 Ω 和函数集合 F 构成的世界模型，文档生成通过组合属性语法（CAG）形式化描述。

要素一

概率上下文无关文法（PCFG）

通过非终结符的递归推导生成语法树 τ。

要素二

产出操作 Y

把语法树映射为字符串序列，同时携带属性传递机制。

$$Y\left(\psi\left[\tau_1, \cdots, \tau_\ell\right], \left\langle x_1, \cdots, x_a \right\rangle, r\right)$$

每个非终结符节点关联属性元组 $\langle x_1, \ldots, x_a \rangle \in \Omega^a$，$a$ 为该节点的元数；随机源 r 控制子节点顺序、重复次数等非确定性操作，使单条生成规则可对应多种表层结构。

例如“capital(x) = y”的语义关系可通过不同语法结构实现，展现出语言生成中的结构重组能力。

迭代复杂性与学习界限

定义迭代复杂度 R_n 为生成 n 次重复操作所需的最小描述长度增量。

$$D\left[\tau\right] \leqslant R_n + D\left[\theta\right] + \frac{1}{\rho}\log\left[p_\tau \cdot C\left(\Omega, n\right)\right]$$

存在语法树 τ 使其产出包含 $\theta(x_1), \ldots, \theta(x_n)$；$p_\tau$ 为生成概率，$C(\Omega, n)$ 为从 Ω 中选取 n 个不同对象的组合数。

$$P_m = \overline{x_1\phi(x_1)}\, s\, \overline{x_2\phi(x_2)}\, s \cdots s\, \overline{x_{m-1}\phi(x_{m-1})}\, s\, \overline{x_m}$$

$$L \leqslant O\left(R_n + D\left[\tau_\phi\right]\right)$$

函数 $\phi : \Omega \to \Omega^d$ 由描述长度 $D[\tau_\phi]$ 的语法树实现；含 m 个示例的提示序列 P_m 上，理想预测器 M 的累积错误率上界如上。

当语法包含循环结构时（如自然语言中的并列句式），R_n 可保持为常数级别，这为长距离结构依赖提供了理论基础。

依据 《大语言模型：训练、精调与对齐》第 6.9.3 节

结构假设的作用

证明的核心思想是贝叶斯模型比较：生成提示 P_m 存在两种竞争假设。

独立生成假说

把每个 $\overline{x_i\phi(x_i)}$ 视为独立事件，其联合概率随 m 指数衰减。

结构生成假说

把整个提示视为单一组合操作的重复应用，其概率由生成树复杂度决定。

$$D\left[\tau_\phi\right] + R_n < m \log |\Omega|$$

满足该条件时，结构假说占据主导，使模型选择延续当前模式。

- 这揭示了 ICL 能力的本质：当预训练数据中组合结构的描述效率 $D\left[\tau_\phi\right]$ 与迭代效率 R_n 足够高时，模型可通过识别生成模式而非记忆具体内容来实现泛化。
- 换言之，示例越多，结构解释的描述长度优势越明显。

思维链的理论解释

对复合函数 $\phi = \phi_1 \circ \phi_2$，直接提示需要构造 $D[\tau_\phi]$ 级别的描述树；思维链提示把它分解为两个步骤。

首阶段

生成中间变量

提示生成中间变量 $\phi_1(x)$，错误界为 $O(R_n + D[\tau_{\phi_1}])$。

次阶段

基于中间结果求解

基于已知的 $\phi_1(x)$ 生成最终结果，错误界为 $O(R_n + D[\tau_{\phi_2}])$。

$$L \sim \max\left(D\left[\tau_{\phi_1}\right], D\left[\tau_{\phi_2}\right]\right) \quad \text{而非} \quad D\left[\tau_{\phi_1}\right] + D\left[\tau_{\phi_2}\right]$$

当中间变量显式出现时，两个阶段的假设空间解耦，各自只需匹配对应层级的生成结构。

这从理论上解释了实践中思维链提示对复杂任务的有效性提升：分解把“一次解析整棵树”变成“两次解析较浅的树”。

MDL 机制

$$\arg\min_{\tau}\left[-\log p\left(d \mid \tau\right)+\rho \cdot D[\tau]\right]$$

从最小描述长度（MDL）原则看，理想预测器 M 隐式执行该优化；d 为观察到的提示，τ 为候选生成结构。

$$n>\frac{D\left[\tau_{\phi}\right]+R_{n}}{\log |\Omega|}$$

组合性结构的压缩能力：描述 n 个实例的模式仅需 $O(D[\tau_{\phi}]+R_{n})$ 比特，独立描述则需 $O(n\log|\Omega|)$ 比特。超过该阈值后，结构解释的概率优势呈指数增长。

- 该机制与 Solomonoff 归纳的普遍性原理相通，但受限于自然语言的结构特性。
- 语法形式主义把无限的潜在假设空间约束为有限复杂度的可生成结构，使 MDL 优化在有限计算资源下可行。
- 这种结构化偏置，正是大规模语言模型在有限数据下展现组合泛化能力的关键。

依据 《大语言模型：训练、精调与对齐》第 6.9.3 节（信息论机制）

思考题与实践题

1 提示词工程被称为“外部优化”。它与有监督微调在改变模型输出分布的方式上有何本质不同？各自代价是什么？

2 用同一道翻译题写出零样本、单样本和少样本三种提示。示例数量增加时，模型依赖的信息发生了什么变化？

3 图 6.2 中零样本与 CoT 提示的结果不同。**仅由示例的写法**解释这一差异，并说明为什么“只给答案”的示例难以奏效。

4 自一致性需要采样多条推理链并投票。若模型的错误答案恰好高度一致，该方法会怎样失效？如何从解码参数上缓解？

5 智能体中，什么是循环（loop）工程、什么是约束（harness）工程？两者各自解决什么问题？请各做一个最小实现。

6 RAG 的检索相关性挑战与生成质量挑战分别对应流程中的哪一步？Self-RAG 用什么信号区分它们？

7 实现一个 ReAct 智能体：给定一个知识源和一组工具，让模型交替进行推理与行动，直至给出最终答案。

8 实现基于 LLM 的工具调用（如调用命令行）：由模型决定何时调用、传什么参数，并把执行结果拼回上下文。

说明 使用 AI 编程（vibe coding）完成作业。